The Problem Docker Solves

Before containers existed, deploying software was chaotic.

Imagine you build a backend API.

Your machine:

Node 18
MongoDB 7
Ubuntu
OpenSSL version X
LibC version Y

Production server:

Node 16
MongoDB 5
Debian
Different system libraries

Your code runs locally but crashes in production.

This problem happens because **software depends on the environment it runs in**.

An application is not just:

code

It also depends on:

runtime
system libraries
filesystem structure
environment variables
network configuration

Traditionally we tried solving this with **Virtual Machines**.

Virtual Machines

A Virtual Machine (VM) runs a full operating system inside another machine, so it behaves like a complete computer. Because it includes the whole OS and kernel, it is heavier and uses more resources.

Containers

A container only packages the application and its dependencies while sharing the host system's kernel. That makes it much lighter and faster to start.

Why containers?
They make apps portable and consistent—the same container runs the same way on any machine, which solves the classic *"it works on my machine"* problem.

How docker container works with Linux

Docker works mainly because of a few Linux kernel features.

Namespaces give isolation. They make a process think it has its own system — its own processes, network, filesystem, etc. Even though it's actually sharing the same host OS.

cgroups (control groups) limit resources. They control how much CPU, memory, and disk a container can use so one container doesn't take everything.

Union filesystems allow layered images. This lets Docker stack filesystem layers so containers can share common layers and stay lightweight.

So in simple terms:
Docker containers exist because Linux provides built-in isolation and resource control features, and Docker uses those to run apps in separated environments without needing a full virtual machine. 🚀

Docker Architecture

Docker architecture is basically three main parts working together.

First is the Docker Client. This is the command line you use, like when you run docker build or docker run. It doesn't actually run containers itself — it just sends requests.

Second is the Docker Daemon (dockerd). This is the background service that does the real work. It builds images, runs containers, manages networks and volumes. The client talks to this daemon through a REST API.

Third is the Docker Registry. This is where Docker images are stored. When you run a container and the image isn't on your machine, Docker pulls it from a registry like Docker Hub.

So the flow is simple:
You type a command in the Docker Client → it talks to the Docker Daemon → the daemon pulls images from a Registry and runs containers. 🐳

Docker HandsOn (Running first docker container)

1. Install Doker desktop
2. docker --version

If Docker is installed correctly, it will show the version.

3. Your First Container

Run:

docker run hello-world

What happens internally:

1 → CLI sends request to daemon
2 → daemon checks local images
3 → image not found
4 → daemon pulls image from Docker Hub
5 → daemon creates container
6 → container runs program
7 → program exits

You should see a message explaining Docker works.

Docker Image

A Docker image is a read-only package that contains everything needed to run an application — the code, runtime, libraries, dependencies, and configuration.

It acts like a blueprint. When you run the image, Docker creates a container, which is the actual running instance of that image.

Inspecting the Image

Now run:

```
docker images // list all the local images available
```

Example output:

```
REPOSITORY    TAG       IMAGE ID       SIZE
hello-world   latest    abc123         13kB
```

Each image has:

repository name
tag (version)
image id
size

A Docker image is a read-only package that contains everything needed to run an app:

- code
- runtime (Node, Python, etc.)
- dependencies
- system libraries

It is used to create containers.

Image = blueprint
Container = running instance

You can run **many containers from one image**.

Example:

```
docker run node:18
```

Docker creates a container from the node:18 image.

Images Are Made of Layers

A Docker image is **not one big file**.
It is a **stack of layers**.

Example structure:

Layer 4 → your app code
Layer 3 → npm install dependencies
Layer 2 → Node runtime
Layer 1 → Linux base system

Docker stacks these layers together to create the final filesystem.

Why Layers Matter (Important for Devs)

1. Faster Builds (Caching)

Each Dockerfile instruction creates a **layer**.

Example:

```
FROM node:18
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
```

If you rebuild the image and **only code changes**, Docker **reuses previous layers**.

So it skips npm install → builds much faster.

. Shared Storage

Many images share the same base layer.

Example:

node:18
python:3.11
ruby:3.2

They might all use the **same Linux base layer**.

Docker stores it **once**, saving disk space.

Docker container

A Docker container is a running instance of a Docker image. It is an isolated environment where the application actually executes.

It contains the app and its dependencies, runs as a process on the host machine, and shares the host OS kernel while staying isolated from other containers.

Docker Image = packaged application template
Docker Container = running instance of that template 🫡

Container Lifecycle

Containers have a simple lifecycle:

created → running → stopped → removed

Containers Are Ephemeral

Containers are meant to be temporary.

If a container is deleted:

- files created inside it
- logs
- runtime changes

are lost.

That's why Docker provides volumes for persistent data.

Basic command to Run docker container

docker run -p hostPort:containerPort imageName

Port mapping (-p) connects a port on your computer to a port inside the container so you can access the app.

docker run -p 3000:3000 node-app

This means:

- Left side (3000) → port on your local machine
- Right side (3000) → port inside the container

So when you open:

localhost:3000

the request goes from your machine → into the container → to the app running on port 3000. 🚀

Run container

docker run nginx

List running containers:

docker ps

List all containers:

docker ps -a

Stop container

docker stop container_id

Delete container

docker rm container_id

Running Containers in Background

Normally containers attach to the terminal.

Run in **detached mode**:

docker run -d nginx

Now it runs in the background.

Naming Containers (Useful)

Instead of using random IDs:

docker run --name my-api nginx

Now you can manage it easily:

docker stop my-api

Execute Commands Inside Containers

Sometimes you need to debug inside a container.

docker exec -it container_name bash

Now you are **inside the container shell**.

Very common for debugging.

Running Queries inside container

Using images in app

Run **PostgreSQL in Docker** and execute SQL queries using psql.

This teaches you:

- pulling images
- running containers
- port mapping
- entering container shell
- using a service inside Docker

1️⃣ Pull the PostgreSQL Image

docker pull postgres

This downloads the **PostgreSQL image** from Docker Hub.

Check:

docker images

2️⃣ Run the PostgreSQL Container

```
docker run -d \
-p 5432:5432 \
-e POSTGRES_PASSWORD=secret \
--name my-postgres \
postgres
```

Important parts:

- -d → run in background
- -p 5432:5432 → port mapping
- -e → environment variable
- --name → container name
- postgres → image

Check container:

docker ps

3️⃣ Enter PostgreSQL CLI

Instead of going into bash, go **directly into psql**:

docker exec -it my-postgres psql -U postgres

Now you're inside:

postgres=#

4️⃣ Run Some SQL

Create database

You can direct use run command docker will automatically pull the images for it if not exist

Volumes

Containers are ephemeral (temporary).

If a container is deleted:

- files created inside it
- logs
- database data

all disappear.

Example:

You run Postgres in a container → store data → delete container → data gone.

So Docker provides persistent storage.

When you attach a **Docker volume**, the data is stored outside the container. If you delete the container with docker rm, the **volume and its data remain safe**.

If you start another container with the **same volume**, it will **reuse the existing data**. The data is deleted only if you **manually remove the volume**.

1️⃣ Container Storage (Default)

If you don't configure anything, data lives **inside the container layer**.

Container
└── writable layer (data stored here)

Problem:

If the container is removed:

docker rm container

💥 Data is lost

So this is **not used for databases**.

2️⃣ Docker Volumes (Most Common)

A **Docker volume** stores data **outside the container** but managed by Docker.

Container
↓
Docker Volume
↓
Host disk

Example:

```
docker run -d \
-v postgres-data:/var/lib/postgresql/data \
postgres
```

Meaning:

volume → postgres-data
container folder → /var/lib/postgresql/data

Now:

- delete container ✓
- run new container ✓
- **data still exists**

Best for:

- databases
- production apps
- long-term storage

Inside the Container

For the official **PostgreSQL image**, the database always stores data at:

/var/lib/postgresql/data

This path is **inside the container filesystem**.

Each Image Has Its Own Data Path

Every Docker image defines **where it stores its data inside the container**.

Examples:

PostgreSQL

/var/lib/postgresql/data

Redis

/data

MySQL

/var/lib/mysql

When Does the Data Actually Get Deleted?

Only if you remove the volume:

docker volume rm postgres-data

or prune unused volumes:

docker volume prune

3️⃣ Bind Mounts (Dev Environment)

Bind mounts connect a **specific host folder** to the container.

Example:

docker run -v ./app:/app node

Meaning:

your laptop folder → container folder

If you edit code locally → container sees it instantly.

Best for: development, hot reload, editing code

Networks

1️⃣ Containers Are Isolated

Each container runs in its own network environment.

So by default:

Container A ❌ cannot talk to Container B directly

They need to be on the same Docker network.

2️⃣ Port Mapping (Access From Your Laptop)

When you run:

docker run -p 3000:3000 node-app

It means:

localhost:3000 → container:3000

Your **browser / local machine** can now reach the container.

This is mainly for **external access**.

3️⃣ Container-to-Container Communication

If two containers are on the **same Docker network**, they can talk using **container names**.

Example:

Node container → API
Postgres container → DB

Node connects like this:

postgres://postgres:password@db:5432/mydb

db = container name.

No localhost here.

4️⃣ Creating a Network

Create one:

```
docker network create my-network
```

Run containers on it:

```
docker run -d --name db --network my-network postgres
```

```
docker run -d --name api --network my-network node-app
```

Now:

api → db:5432

works automatically.

Docker provides **built-in DNS**.

5️⃣ Check Networks

List networks:

```
docker network ls
```

Inspect one:

```
docker network inspect my-network
```

You'll see connected containers.

Real Backend Setup

Typical architecture:

Node API container
↓
Postgres container
↓
Redis container

All connected via **same Docker network**.

Node connects using:

postgres://db:5432
redis://redis:6379

One Rule to Remember

Host machine → use localhost + port mapping
Container → container → use container name

Remove your custom network (my-network):

docker network rm my-network

What Matters for You

You mainly need to know:

- -p → expose container to your machine
- docker network create
- containers communicate using **names**

That's honestly **most Docker networking in real dev work**.

Dockerise Your app

1️⃣ Dockerfile (Express App)

Location: project root (next to package.json)

```
FROM node:22-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

EXPOSE 3000

CMD ["node", "src/index.js"]   # or "ts-node src/index.ts" if TS
```

Key points:

- **WORKDIR /app → avoids using / root**
- **COPY package*.json → allows layer caching**
- **EXPOSE 3000 → app port**
- **CMD → start app**

Then run comment in dockerfile folder

```
docker build -t . imagename .
```

your image is ready

Docker Compose

What is Docker Compose?

- Docker Compose lets you define multiple containers in one YAML file.
- You can start, stop, and manage your app stack with one command (docker-compose up).
- Think of it as “Docker for your entire app” — Node.js, MongoDB, Redis, etc.

🔹 Key Concepts

1. Service → A container you want to run (e.g., app, mongo)
2. Image → The Docker image to use (e.g., node:20, mongo:latest)
3. Volumes → Persist data outside the container
4. Networks → Allow containers to talk to each other by service name

🔸 Minimal Example: Node.js + MongoDB

Create a docker-compose.yml in your project root:

```
version: '3.9'

services:
  mongo:
    image: mongo:latest
    container_name: mongo
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db
```

```
  app:
    build: .
    container_name: my-app
    ports:
      - "3000:3000"
    depends_on:
      - mongo
    environment:
      - MONGO_URI=mongodb://mongo:27017/myDatabase
    networks:
      - app-network

volumes:
  mongo-data:

networks:
  app-network:
```

How this works

- mongo service runs MongoDB and stores data in mongo-data volume.
- app service runs your Node.js app, waits for mongo to start (depends_on), and uses MONGO_URI to connect.
- Both services are on the same network app-network, so mongo can be accessed **by name**.

Start your stack

docker-compose up -d

- -d → detached mode (runs in background)
- To see logs: docker-compose logs -f
- Stop everything: docker-compose down (keeps volumes unless you add -v)